

Bonus Training Proposal

面向桌面抓取的视觉-语言条件运动生成

1 项目目标

在 Track 4 基础任务之外, 我们计划增加一个真正包含 **simulation data collection**、**model training** 和 **generalization evaluation** 的 Bonus Extension。

基础任务仍保持不变:

抓取并抬起桌面上的方块

机器人不进行 locomotion, 保持站立状态; 桌面、相机位置和机器人初始姿态固定。Bonus 的主要变化是:

方块位置随机变化

模型需要根据当前 camera image 和自然语言指令, 自主生成能够被 SONIC 执行的 G1 运动轨迹。

核心问题为:

能否训练一个轻量级 Vision-Language Conditioned Motion Generator, 根据单帧视觉输入和文本指令, 生成 SONIC-compatible 的 G1 抓取轨迹, 并泛化到训练集中未见过的方块位置?

2 任务定义

模型定义为:

$$f_{\theta}(I, L) \rightarrow Q_{1:T}^{\text{upper}}$$

其中:

$$I \in \mathbb{R}^{H \times W \times 3}$$

表示 simulated head-mounted camera 获取的一张 RGB 图像；

$$L$$

表示自然语言指令，例如：

``Pick up the red block.``

模型输出：

$$Q_{1:T}^{\text{upper}}$$

即未来一段时间内 G1 upper-body 的 reference joint trajectory。

机器人下半身不由网络生成，而采用固定 standing reference：

$$Q_{1:T}^{\text{lower}} = Q^{\text{standing}}.$$

最终完整轨迹为：

$$Q_{1:T}^{\text{ref}} = [Q_{1:T}^{\text{lower}}, Q_{1:T}^{\text{upper}}].$$

因此整体 inference pipeline 为：

$$(I, L) \rightarrow Q_{1:T}^{\text{upper}} \rightarrow Q_{1:T}^{\text{ref}} \rightarrow \text{SONIC} \rightarrow \text{G1}$$

3 模型输入

第一版模型仅使用两个主要输入。

3.1 RGB Image

输入为当前时刻的一张 head-camera RGB image：

$$I_t \in \mathbb{R}^{H \times W \times 3}.$$

视觉信息主要用于获取：

目标方块的位置及其与机器人之间的空间关系

由于机器人初始姿态固定，因此第一版不必显式输入 robot state。

如果后续进一步随机化机器人初始姿态，则可以扩展为：

$$f_{\theta}(I, L, q_t, \dot{q}_t).$$

3.2 Language Instruction

文本输入用于指定当前 manipulation goal，例如：

```Pick up the red block.```

如果场景中始终只有一个方块，并且指令永远是

```Pick up the block.```

那么 language 实际上是常量，模型可以完全忽略文本输入。

因此为了让 Vision-Language conditioning 真正成立，可以在场景中设置两个可区分方块，例如：

red block blue block

对应指令：

```Pick up the red block.```

或：

```Pick up the blue block.```

任务本身仍然只有：

grasp + lift

但输出轨迹真正依赖：

$$p(Q | I, L).$$

4 模型输出与 SONIC 接口

网络主要预测 upper-body joint trajectory:

$$Q_{1:T}^{\text{upper}}.$$

下半身使用固定 standing motion:

$$Q_{1:T}^{\text{lower}} = Q^{\text{standing}}.$$

二者组合得到:

$$Q_{1:T}^{\text{ref}} \in \mathbb{R}^{T \times 29}.$$

即完整的 G1 29-DoF reference joint position。

对应的 joint velocity 可以通过位置差分计算:

$$\dot{q}_t^{\text{ref}} = \frac{q_t^{\text{ref}} - q_{t-1}^{\text{ref}}}{\Delta t}.$$

对于 stationary grasping, root position 和 root orientation 可以采用固定 standing reference。

因此送入 SONIC 的 reference motion 可以统一表示为:

$$(Q^{\text{ref}}, \dot{Q}^{\text{ref}}, R^{\text{root}}, P^{\text{root}})$$

其中:

$$Q^{\text{ref}} \in \mathbb{R}^{T \times 29},$$

$$\dot{Q}^{\text{ref}} \in \mathbb{R}^{T \times 29}.$$

Hand / gripper control 可以作为单独的输出 branch:

$$G_{1:T}.$$

最终:

$$\text{Network} \rightarrow \text{SONIC reference motion} \rightarrow \text{SONIC} \rightarrow \text{G1 grasp}$$

5 网络结构

由于场景仅包含桌面、少量方块和机器人手臂，视觉理解难度远低于自动驾驶等复杂环境，因此没有必要使用很重的 perception backbone 或 BEV architecture。

建议采用：

Pretrained Vision Encoder + Text Encoder + Small Fusion Transformer + Motion Decoder.

5.1 Vision Encoder

候选方案：

DINOv2 ViT-S/14

或其他轻量 pretrained ViT。

建议：

- 参数量约 20-25M；
- 第一阶段冻结 Vision Backbone；
- 不仅使用 global CLS token；
- 保留 spatial patch tokens。

即：

$$I \xrightarrow{\text{Vision Encoder}} Z_v$$

其中：

$$Z_v \in \mathbb{R}^{N \times D}.$$

保留 patch-level representation 的原因是，grasping 不仅需要知道：

“图像中存在一个 block”

还需要知道：

“block 位于什么位置”。

5.2 Text Encoder

文本部分不需要大型 LLM。

可以采用 lightweight pretrained text encoder, 例如 MiniLM-class model:

$$L \xrightarrow{\text{Text Encoder}} Z_l.$$

第一阶段同样冻结 pretrained text encoder。

5.3 Multimodal Fusion

视觉 token 和文本 token 统一投影到:

$$d_{\text{model}} = 256.$$

然后使用一个小型 Transformer 进行 multimodal fusion。

推荐初始配置如下:

模块	配置
Vision Encoder	DINOv2 ViT-S/14, Frozen
Text Encoder	MiniLM-class, Frozen
Hidden Dimension	256
Fusion Transformer	4 Layers
Attention Heads	8
FFN Dimension	1024
可训练参数量	约 5–10M

整体 representation pipeline 为:

$$I \rightarrow Z_v$$

$$L \rightarrow Z_l$$

$$(Z_v, Z_l) \rightarrow Z_{\text{condition}}.$$

6 Motion Decoder

Motion Decoder 可以分为两个阶段实现。

6.1 方案 A: Deterministic Transformer Regression

第一版使用最简单、风险最低的 Transformer trajectory decoder。

模型直接预测：

$$(Z_v, Z_l) \rightarrow \hat{Q}_{1:T}.$$

可以设置一组 learned trajectory queries：

$$e_1, e_2, \dots, e_T$$

分别对应未来各个 timestep。

Joint position loss：

$$\mathcal{L}_{\text{pos}} = \frac{1}{T} \sum_{t=1}^T \|\hat{q}_t - q_t^{GT}\|_1.$$

为了保证 trajectory smoothness, 可以增加 velocity consistency loss：

$$\mathcal{L}_{\text{vel}} = \frac{1}{T-1} \sum_{t=2}^T \|(\hat{q}_t - \hat{q}_{t-1}) - (q_t^{GT} - q_{t-1}^{GT})\|_1.$$

总 loss：

$$\mathcal{L} = \mathcal{L}_{\text{pos}} + \lambda \mathcal{L}_{\text{vel}}.$$

6.2 方案 B: Flow Matching Motion Generation

如果 deterministic baseline 能稳定工作, 可以进一步将 motion decoder 替换为 Flow Matching / DiT-style generator。

模型学习：

$$v_\theta(Q_\tau, \tau, Z_{\text{condition}}).$$

轨迹生成过程为：

$$\frac{dQ_\tau}{d\tau} = v_\theta(Q_\tau, \tau, Z_{\text{condition}}).$$

从简单噪声分布开始：

$$Q_{\tau=0} \sim \mathcal{N}(0, I)$$

最终生成：

$$Q_{\tau=1} = Q_{1:T}^{\text{ref}}.$$

因此可以进一步比较：

Deterministic Transformer Regression

与：

Flow-Matching Motion Generation

是否在 motion quality、generalization 和 inference speed 上存在差异。

第一版不需要加入 Proposal Transformer。对于当前单一 tabletop grasping task，直接进行 conditional trajectory generation 更简单，也更容易控制实验变量。

7 Simulation Data Collection

训练数据直接在 simulation 中自动生成。

每一个 episode 可以按照以下流程构建：

1. 将 G1 reset 到固定 standing initial pose；
2. 在桌面可达区域随机采样 block position；
3. render 当前 head-camera RGB image；
4. 使用 expert controller 生成 grasp-and-lift trajectory；
5. 在 simulation 中执行轨迹；
6. 检查是否成功抓取并抬起；
7. 保存成功 episode。

单个训练样本可以表示为：

$$D_i = (I_i, L_i, Q_i^{GT}, G_i^{GT})$$

其中：

- I_i : head-camera RGB image;
- L_i : language instruction;
- Q_i^{GT} : expert joint trajectory;
- G_i^{GT} : hand/gripper command.

8 Expert Trajectory Generation

训练所需要的 Ground Truth trajectory 可以由 simulation 自动产生。

第一选择是：

Scripted IK / Motion Planner

例如：

initial pose $\rightarrow$ pre-grasp pose $\rightarrow$ grasp pose $\rightarrow$ lift pose.

原因是 scripted IK trajectory：

- 几何关系明确；
- grasp target 精确；
- 数据生成成本低；
- 容易自动生成大量训练数据；
- 不会额外引入 pretrained motion generator 的误差。

如果 Kimodo 或 ARDY 能稳定生成高质量 grasp motion，也可以作为 additional demonstration source。

9 训练数据的泛化设计

训练集和测试集需要在 block position 上分离。

例如桌面可达空间定义为：

$$x \in [x_{\min}, x_{\max}], \quad y \in [y_{\min}, y_{\max}].$$

训练时随机采样一组位置：

$$\mathcal{P}_{train},$$

测试时使用不同位置:

$$\mathcal{P}_{test},$$

满足:

$$\mathcal{P}_{train} \cap \mathcal{P}_{test} = \emptyset.$$

因此真正评估的是:

是否能根据视觉输入泛化到 unseen block positions

而不是简单记忆固定轨迹。

10 Evaluation

主要 evaluation metrics 包括:

10.1 Grasp Success Rate

定义为:

$$\text{Success Rate} = \frac{\text{成功抓取并抬起的 episode 数}}{\text{总 episode 数}}.$$

10.2 Target Selection Accuracy

如果存在多个方块, 则检测机器人是否抓取 language instruction 指定的目标。

10.3 Motion Precision

计算 end-effector 与期望 grasp pose 的误差:

$$E_{EE} = \|p_{EE} - p_{\text{target}}\|_2.$$

10.4 Physical Feasibility

统计:

- joint-limit violation;
- collision;
- unstable motion;
- SONIC tracking failure;
- grasp/contact failure。

10.5 Inference Latency

测量:

camera input $\rightarrow$ complete motion available

所需要的 inference time。

10.6 Generalization

分别报告:

Seen-position Success Rate

和:

Unseen-position Success Rate.

11 可选 Ablation Study

如果时间和算力允许, 可以进一步做以下受控实验:

1. Vision backbone frozen vs. partial fine-tuning;
2. global visual embedding vs. spatial patch tokens;
3. Vision-only vs. Vision-Language conditioning;
4. Transformer Regression vs. Flow Matching;

5. seen block positions vs. unseen block positions。

不要求全部完成。优先保证完整的：

data $\rightarrow$ training $\rightarrow$ SONIC execution $\rightarrow$ evaluation

pipeline。

12 预期贡献

该 Bonus Extension 的主要贡献不是增加新的机器人任务，而是在固定 grasp-and-lift task 上增加：

1. simulation dataset generation；
2. vision-conditioned motion generation；
3. language-conditioned target selection；
4. trainable trajectory generation；
5. unseen object position generalization；
6. SONIC-compatible motion execution。

整体 pipeline 为：

Simulation Data Collection $\rightarrow$ Vision-Language Encoding $\rightarrow$ Motion Generator Training $\rightarrow$ SONIC $\rightarrow$ Generalization

最终目标可以概括为：

Vision-Language Conditioned Motion Generation for Generalized Tabletop Grasping

即：

Image + Text $\rightarrow$ G1 Reference Motion $\rightarrow$ SONIC $\rightarrow$ Grasp and Lift